

Finite-sample decision risk in unseeded quantum compiler benchmarking

Panagiotis Gkilis · BedVibe Studios, Oslo, Norway · bedvibe@bedvibe.studio ORCID 0009-0007-3805-170X

Version 4, 2026-09-13. Not submitted. All data and code: [qvalidation repository](#).

Abstract

Quantum compiler benchmarks are used to accept or reject changes to production transpilers, but the compilers they measure are stochastic. We show that at the pinned revision of IBM's Benchpress benchmark suite, its Qiskit gym compiles without passing `seed_transpiler`, so every gate count it reports is one draw from an unmeasured distribution — while the BQSKit gym in the same repository does seed its compiler. We quantify the consequence as **finite-sample decision risk**: the probability that a regression verdict computed from k runs per version disagrees with the verdict implied by the same measurement's long-run mean.

In a pre-registered study of 39 QASMBench circuits (28–420 qubits) transpiled to a heavy-hex lattice under Qiskit 1.4.3 and 2.0.0, using 200 distinct transpiler seeds per arm — the smallest 200 of 400 draws, so lower order statistics rather than an IID full-range sample (§3.4) — across ten operating-system processes, we find that circuits whose compilation is deterministic carry zero risk by construction, while among the 23 resolved circuits whose compilation is stochastic the risk falls with distance from the decision boundary — an association we report descriptively and do not attribute — thresholding can induce a relationship of this kind, but the quantitative claim that it *largely* does was withdrawn in v3 with the synthetic null that supported it, and is not reinstated (§4.1). Seven of 26 eligible circuits carry a risk of at least 5% and four carry at least 10% under a +10% threshold. The median circuit admits an **ambiguity band of 10.9 percentage points** — the total width of the interval over which the three-run rule's call probability runs from 5% to 95%. The band is constructed from the measured residuals of **this** version pair and is not independent of it.

Two of the three circuits named in Qiskit issue #14402 are affected. On a matched `linear` topology we reproduce that issue's two low-variance figures to within 0.63 and 0.29 percentage points, and show that its remaining figure (+46.14% on `bv_n140`) sits at the 87.8th percentile of the distribution a three-run comparison produces **at our configuration**, whose exact support runs from −17.68% to +109.17%. The issue's historical Benchpress revision and aggregation rule are not supplied, so these are numerical comparisons at our configuration, not a reproduction of its protocol, and the distribution above is ours rather than the one its protocol would have produced.

We do not claim a suite-level failure rate; our sample is underpowered for one. We claim that finite-sample decision risk is measurable and reproducible on a benchmark suite used to evaluate a production compiler, under an explicitly hypothetical +10% threshold and aggregation policy that the suite itself does not define. We identify no compiler mechanism, and we show no published decision to have been wrong.

1. Introduction

Compiler benchmarks answer a binary question: is this change a regression? For classical compilers the measurement is nearly deterministic and the question is nearly settled by one run. For quantum compilers it is not. Routing a circuit onto a constrained qubit topology is NP-hard, and production transpilers use stochastic heuristics — Qiskit’s SABRE among them. The same circuit, the same compiler and the same target produce different gate counts on different runs.

This is well known. What has not been measured is what it costs the decisions built on top of it.

This paper is also the first output of a validation apparatus we are building, and it is written to expose that apparatus rather than only its result. The measurement pipeline, the pre-registration mechanism, the numeric inventory that re-checks every published figure against a fresh recomputation, the replication artifact, and the adversarial audit are all part of the contribution. **The Qiskit experiment is the demonstration case; the reusable object is the method for quantifying decision risk in a stochastic benchmark.** Nothing in the apparatus is specific to Qiskit: it requires only a benchmark whose observable is a scalar, a source of run-to-run variation that can be enumerated or sampled, and a decision rule.

Our contribution is threefold. First, a source-level finding: **at the pinned revision, the Qiskit gym of IBM’s Benchpress suite compiles without passing `seed_transpiler`**, and the inspected QASMBench gate-count path does not aggregate repeated transpilation counts before returning the measured circuit count (§2.1), so a reported figure is one sample and the choice of estimator is left entirely to the reader. This is a claim about one gym at one revision and not about the suite as a whole: the sibling BQSKit gym does seed its compiler, and the remaining six gyms are not assessed here (§2.1). Second, a definition and measurement: **finite-sample decision risk**, the probability that a k -run verdict disagrees with the long-run verdict of the same measurement. Third, a pre-registered experiment that measures this risk on 39 circuits and reports what predicts it — while declining to call that association a compiler mechanism (§4.1).

We are deliberately narrow about what follows. We do not claim any published decision was wrong, we do not attribute anything to a specific commit, and we withdraw — in §5 — a suite-level rate that an earlier draft of this work reported.

2. Background

2.1 Benchpress

Benchpress (Nation et al., *Nature Computational Science*, 2025; doi:10.1038/s43588-025-00792-y) compiles a corpus of circuits against abstract topologies and records circuit properties. Its 2-qubit gate observable is, verbatim from `benchpress/qiskit_gym/utils/io.py`:

```
benchmark.extra_info["output_gate_count_2q"] = circuit.count_ops().get(two_qubit_gate, 0)
```

Two properties matter here, both established by reading the source at commit `b695f30e83a32bac05b9b4d8e98d37ba9aae5236`:

1. **The Qiskit gym compiles unseeded — and a sibling gym does not.** At commit `b695f30e`, the Qiskit-based gyms call `generate_preset_pass_manager(optimization_level=..., backend=...)` with

no `seed_transpiler` anywhere. Circuit-construction seeds are pinned (`seed=12345`, 14 occurrences); the compilation seed is not.

△ We state this for the Qiskit gym only. `seed_transpiler` and `generate_preset_pass_manager` are **Qiskit-specific APIs**; the other six gyms cannot call them and use their own interfaces — TKET, for instance, calls `backend.default_compilation_pass(optimisation_level=...)`. More importantly, the △ v4 correction (Astra A11), two scope limits on this paragraph. The unseeded finding is about the **ordinary preset-pass-manager path**; the separate qiskit-transpiler-service gym calls `TranspilerService` and is a different compilation path, not covered by it. And the BQSKit statement below is about the **inspected files**, not about every BQSKit path in the suite. The **BQSKit gym explicitly seeds its compiler**: `bqskit_gym/device_transpile/test_summit.py:179` calls `compile(circuit, model=BACKEND, optimization_level=..., compiler=compiler, seed=0)`. **Whether the remaining SDKs compile deterministically is not assessed here.** This is a claim about one revision of one repository, not a permanent property of Benchpress.

2. **No aggregation of repeated gate-count measurements exists on the inspected path.** The gate count is read from one returned circuit: no mean, no minimum, no best-of- k . △ v3 correction: the earlier wording “no aggregation exists anywhere” and “no run-count option” was too broad. Benchpress drives its workouts with `pytest-benchmark` and sets `--benchmark-min-rounds=1`; that framework does support repetition and aggregation, of **timings**. No gate-count aggregation is implemented, which is the claim the argument needs.

Consequently the aggregation is the user’s choice, and different users may choose differently.

2.2 Qiskit issue #14402

In May 2025 the lead author of Benchpress reported large 2-qubit gate count increases between Qiskit 1.4.3 and 2.0, writing: “*This was verified by running Benchpress several times for each version to get statistics. E.g. over the full test suite the values returned for 3 runs was:*”. Per-test figures are described as “*the avg. percent increase in 2Q gate counts*”; the basis of that average is not stated. The largest reported cases are `bv_n140-linear` **+46.142%**, `bv_n280-linear` **+44.245%** and `knn_341-linear` **+44.060%**. △ v4 correction: v1–v3 rounded the third of these to +41%, which is not what the issue reports; the error propagated into §4.4’s comparison table and reversed the sign of one difference.

We take $k = 3$ from this report. It is an external choice, not ours, and Benchpress itself offers no such parameter.

3. Methods

3.1 Estimand

For circuit c , topology T and compiler versions A (baseline) and B (candidate), let $X_A(s)$, $X_B(s)$ be the 2-qubit gate counts produced with `seed_transpiler = s`. △ v4 correction: v1–v3 called these “deterministic functions of s ”. Not all relevant randomness is controlled — `FlexibleBackend` draws backend error rates unseeded on every construction — so the defensible statement is that **repeating a fixed seed reproduced the same count in every control we ran**: across processes with differing `PYTHONHASHSEED`, across an independently reconstructed environment, and across two machines with different CPU vendors (§3.4). Backend randomness was probed in a 0-of-24 control, which is a limited negative result and not proof that it never matters. Define the **reference change**

$$\theta = E_s[X_B(s)] / E_s[X_A(s)] - 1$$

estimated by the ratio of arm means. The protocol draws k seeds per arm independently and flags a regression when $(\text{mean}_B - \text{mean}_A) / \text{mean}_A \geq t$. **Finite-sample decision risk** is the probability, over the protocol's own draw space, that its verdict differs from the verdict implied by θ .

θ is an estimated long-run mean of the same stochastic process, not external ground truth. The quantity is therefore the procedure's finite-sample error relative to its own asymptote. This is weaker than an external criterion, and no external criterion exists: the “true” gate count of a stochastic compiler is not defined independently of the compiler. We use the phrase *decision risk*, never *wrong answer*.

3.2 Decision rule, computed exactly

Gate counts are integers, so the rule is exactly decidable. With arm sums S_A, S_B over k draws and $t = p/q$ in lowest terms,

$$(S_B - S_A) / S_A \geq p/q \Leftrightarrow q \cdot S_B \geq (q+p) \cdot S_A$$

which involves no floating point. This matters: the natural float form $b \geq a(1+t)$ is *not* the same expression as $(b-a)/a \geq t$, and disagreed with it in 84 of 900 randomised cases during development. The integer form was verified against exact-integer brute force over 900 cases with zero disagreements. Risk is then computed by enumerating all $200^3 = 8,000,000$ sum-triples per arm for $k \leq 3$, and by Monte Carlo above that.

Two conventions are stated explicitly. **Draws are enumerated with replacement**, so the tuple (s, s) is included; a maintainer running k distinct seeds corresponds to enumeration *without* replacement. We computed both: on the pre-registered data the per-circuit risk differs by at most 0.0011 and the count of circuits at risk $\geq 5\%$ is unchanged at 7. **Equality at the threshold counts as a regression call**, consistent with the $\geq$ in the decision rule. The integer path is unguarded against a zero baseline sum, which is undefined; no circuit in this study has a zero 2-qubit gate count, so the branch is never exercised.

3.3 Pre-registration

The circuit list, protocol, analysis code and falsification criteria were committed before **the first tracked commit of the primary 39-circuit raw data**. Δ v4 correction (Astra V4-05): earlier versions of this sentence said “before the primary raw data existed”, which asserts more than a commit log can support — see the correction below, which this sentence now matches instead of contradicting.

Δ v3 correction. Earlier versions said “before any measurement existed” and called the absence of cherry-picking “a checkable fact”. Both overstate the record. A census and exploratory results **preceded** the pre-registration: commit c426338, earlier the same day, already identifies heavy-hex as the worst case, and the ≤ 10 s selection rule is computed from that pre-existing census. What the commit order supports is a **frozen follow-up design after exploration** — the circuit list and the bv_n140 exclusion were fixed before the primary data was added and did not change afterwards — not blindness to earlier outcomes. Git records the order in which files were committed locally; it does not record when their outputs were first looked at, and local timestamps are not independent evidence of that. Δ v4 correction (Astra A09): git establishes the order in which files were COMMITTED and nothing more. It does not establish when an output was first seen, nor that no uncommitted data existed, nor that local

clock times are truthful. The defensible statement is that the analysis code and circuit list were **committed before the first tracked commit of the primary raw data**, and that per the author’s process records the analysis was intended to be blind. Commit order is verifiable (`git log --diff-filter=A -- prereg_analysis.py results/raw/prereg`):

commit	timestamp	content
a36a34a	2026-09-03 23:51:08	pre-registration
fb573d	2026-09-03 23:52:38	analysis code and circuit list
30224a0	2026-09-04 09:13:07	raw data, unanalysed

Selection was on compute cost only: every `qasmbench-large` circuit whose 12-seed heavy-hex runtime in a pre-existing census was ≤ 10 s. Δ **v3 correction:** earlier versions said runtime “is independent of both θ and the risk”. That is not established and is withdrawn. The defensible statement is procedural — **the implemented selection rule uses runtime, and consulted neither θ nor risk** — with the caveat that exploratory results existed before the rule was frozen (§3.3). This yielded **39 circuits, 28–420 qubits**. One circuit, `bv_n140`, was excluded from the primary analysis because it had been selected post-hoc in earlier exploratory work; it is reported separately in §4.4.

Analysis, fixed in advance. θ interval by percentile bootstrap over seeds, resampled jointly across the paired arms, $B = 4000$. A circuit is `REGRESSION` if that interval lies entirely above t , `NO_REGRESSION` if entirely below, otherwise `UNRESOLVED` and excluded. A circuit whose θ lies within 3 percentage points of t is flagged `BOUNDARY` and excluded. Risk interval by seed bootstrap, $B = 400$. The pre-registered primary endpoint is the proportion of non-boundary, resolved circuits whose risk interval excludes zero.

Δ **v3 correction — what the risk interval conditions on.** Each replicate jointly resamples the seed-indexed arms and recomputes $P(\text{call})$. It does **not** recompute θ ’s side of the threshold, the reference verdict, the `UNRESOLVED` classification, or `BOUNDARY` eligibility, and it does not switch between $P(\text{call})$ and $1 - P(\text{call})$ if θ crosses. These are therefore **conditional bootstrap intervals for $P(\text{call})$ given the observed classification**, not calibrated confidence intervals for long-run decision risk, and earlier versions read them as the latter.

Two things follow. First, a stability result that does hold: across 4,000 resamples, none of the 26 eligible circuits changed θ ’s side of the threshold. Eligibility is materially less stable — `knn_n67` crossed the 3 pp boundary in 22.0% of resamples, `swap_test_n83` in 27.4%, `adder_n118` in 8.6%. Second, a limit that is easy to state and was not stated: coverage is not guaranteed. With arm $A \equiv 100$ and B equal to 100 with probability 0.99 or 2100 with probability 0.01, the population θ is +20% and the true three-run false-negative risk is $0.99^3 = 97.03\%$; yet with probability $0.99^{200} = 13.4\%$ every one of 200 observed draws is 100, and this procedure then reports a non-boundary `NO_REGRESSION` with interval $[0, 0]$. Nominal 95% coverage is at most 86.6% in that construction. **We do not claim the measured compiler populations have such a tail**; the example shows only that a sample of 200 does not by itself answer the coverage question.

3.4 Measurement and controls

Qiskit **1.4.3** $\rightarrow$ **2.0.0**, forward, topology `heavy-hex`, Benchpress pinned at `b695f30e` with module and all 58 circuit hashes recorded. **200 distinct seeds per arm**, realised range 663,193 – 1,105,794,431, zero consecutive pairs, split across **ten OS processes per arm** with differing `PYTHONHASHSEED`. **15,600 transpilations**.

△ **v3 correction — the seed law.** Earlier versions described these as “200 independent transpiler seeds” drawn from $7 \dots 2^{31}$. **They are not an IID sample from that range.** The generator draws **400** integers, deduplicates, sorts, and keeps the **smallest 200** (`scatter.py: sorted(set(rng.integers(7, 2**31 - 1, n * 2))[:n])`). The retained values are therefore dependent lower order statistics.

The signature is visible in the data and is not subtle: **6 of the 200 realised seeds lie above the midpoint of the nominal range, where roughly 100 would be expected** under the claimed uniform draw, and their mean sits at **0.2462** of the range rather than 0.5.

What this does and does not affect. Every reported per-seed measurement stands: the arms are correctly paired, the seeds are distinct, and the risk calculations are exact functions of the recorded values. △ **v4 correction (Astra V4-05):** earlier versions added “each count was reproduced from its seed in the controls described above”, which reads as a claim about all 15,600 observations. The controls are finite and are listed below — **144** counts (6 circuits $\times$ 12 seeds $\times$ 2 versions) reproduced from clean, and **216** per-seed gate counts matched across two machines. Every count *those controls compared* was reproduced exactly; the remaining observations were not re-measured, and the backend randomness noted there applies throughout. What is **not** supported is any claim that the empirical distributions represent the seed space a user would encounter, or that they estimate the behaviour of unseeded operation over the full range. Restoring that would require a rerun under a correctly specified sampling law, which we have not done. The results in §4 are therefore statements about **these** recorded distributions.

Controls, each of which could have invalidated the study:

- **Backend randomness.** `FlexibleBackend` draws error rates unseeded, differently on every construction, and the two arms build backends separately. At optimization level 2 the layout passes score with error rates. Tested with four verified-distinct draws $\times$ six circuits $\times$ four seeds: **0 of 24 cases changed the observable.**
- **Observable identity.** `two_q_gate_type` is `cz` in all three Qiskit versions and on all four topologies; transpiled output contains no other 2-qubit gate.
- **Backend identity.** Coupling map identical across versions (278 edges, hash `ef616023`), as are the basis gates.
- **Seed independence.** Lag-1 autocorrelation of consecutive seeds between -0.22 and $+0.11$; contiguous and scattered seed sets give matching spreads.
- **Determinism.** A fixed seed reproduces the same **2-qubit gate count** across separate OS processes with differing `PYTHONHASHSEED`, across an independently reconstructed clone and environment, and **across two physical machines with different CPU vendors**. The six-circuit replication artifact reproduces **144 of 144** per-seed values from a clean state on both.

△ **v3 correction:** earlier versions said “bit-identically”. The cross-machine records contain gate counts, seeds, versions and topology — **no output QASM hash, operation sequence, layout or final mapping**. Their hashes identify the *input* circuit. What is established is agreement on every compared integer, not identity of the compiled circuits.

The cross-machine check’s protocol was committed before the first tracked commit of any second-machine result (`crossmachine/PREREISTRATION.md`; the commit order is checkable, and establishes commit order only — §3.3) and inherits the frozen six-circuit, twelve-seed selection wholesale, so it

introduces no selection freedom. It covers **all three Qiskit versions**, including the 1.4.3 → 2.0.0 pair the primary study uses. **All 216 per-seed gate counts are identical:**

	machine 1	machine 2
CPU	AMD64 Family 26, AuthenticAMD	Intel64 Family 6, GenuineIntel
cores	16	8
Python	3.10.10	3.10.21

The differing Python patch version was not controlled and cuts in the same direction: more uncontrolled variation, still identical integers.

⚠ **What this does not establish.** ⚠ v4 correction: the cross-machine measurement records do **not** contain a rustworkx version or build — that field was added to the recorder only afterwards. The statement that both machines ran **rustworkx 0.18.1** is a retrospective author assertion, not contemporaneous evidence, and is labelled as such here. Either way the limitation stands and is the stronger reading: because the graph library was not independently recorded, this result cannot separate “*the CPU does not matter*” from “*the graph library was identical, so the CPU never had the opportunity to matter*”. Settling it needs a run that records the build and varies it. The primary 200-seed study itself was executed on **one machine**; only this six-circuit determinism control is cross-machine.

- **Selection bias.** ⚠ v3 correction. The published comparison used 40 circuits against 12, wrongly including the excluded `bv_n140`, and gave $p = 0.7676$. The correct groups — the **39** selected against the **13** complete-census circuits that carry the 12-run data this test needs — give medians **0.1115** and **0.1260**, Mann-Whitney $p = 0.4560$. Six further census circuits lack complete 12-run data and enter neither group. Neither the old nor the corrected test establishes independence: failing to reject a null is not evidence for it, and neither test examines θ or risk directly.

4. Results

4.1 What predicts the risk — and why it is not a mechanism

Of the 26 eligible circuits, **13 compile deterministically** — every seed gives the same gate count in both arms. Their risk is zero *by arithmetic*, not by measurement, and we do not report this as evidence.

Among the **23 resolved circuits whose compilation is stochastic** (note the denominator: 23 of the 36 *resolved* circuits, of which 13 are also non-boundary and therefore *eligible*), risk falls with distance from the decision boundary:

Spearman $\rho = -0.833$ between distance from θ to the threshold and the risk ($p < 0.001$, $n = 23$).

⚠ **This is not a discovered mechanism, and we do not claim it as one.** It is an association measured on 23 circuits, and nothing more.

⚠ **Correction, v3.** v1 and v2 asserted that risk is a monotone decreasing function of $|\theta - t|$ *for any distribution, by the definition of the decision rule. That statement is false and is withdrawn.* A hostile audit supplied an explicit counterexample: with arm A identically 100 and $t = +10\%$, the three-run rule calls when $S_B \geq 330$. If B is identically 109 then $\theta = +9\%$, distance 1 pp, and risk is **0**. If instead B is 200 with probability 0.3 and 29 with probability 0.7 then $\theta = -19.7\%$, distance 29.7 pp, and

a call occurs exactly when at least two of three draws are high — risk **21.6%**. Risk rises from 0 to 21.6% as distance rises from 1 pp to 29.7 pp. The ordering is not universal, and a location-shift argument at fixed shape does not establish a distribution-free theorem.

Our own data violate it too: `adder_n118` sits 3.5301 pp from the cut with risk 12.6743%, while `bv_n280` is *further* away at 5.1102 pp with *higher* risk 17.3147%.

The measured $\rho = -0.833$ is unaffected by this withdrawal and reproduces independently (-0.8329 to four decimals). What is withdrawn is the claim that its sign and magnitude are a distribution-free property of the decision rule.

△ **The synthetic-null figures previously reported here — mean -0.661 over 30 trials, 5 of 30 reaching -0.833 or stronger — are withdrawn.** The audit searched the pinned snapshot and found no generator, parameters, RNG state or trial record that reproduces them, and neither `model_check.py` nor the `followup` scripts implement such a null. We have not regenerated a replacement: producing a fresh simulation now and presenting it as the original evidence would be worse than withdrawing the number. Absent that generator we make no quantitative claim about how much of the association the decision rule induces.

What is empirical is the **scale** of the seed-induced spreads that make the risk non-negligible at a given distance — reported as ambiguity bands in §4.3 — and the fact that 13 of 26 eligible circuits have no spread at all. The apparent clustering of outcomes by algorithm family — `adder`, `knn`, `cc`, `swap_test`, `bv`, `qft` affected; `cat`, `ghz`, `ising`, `wstate` not — is a consequence: the latter families compile deterministically on this topology and therefore cannot err.

4.2 Magnitude

The pre-registered endpoint’s Wilson interval **assumes independent circuits and is therefore too narrow**; a cluster bootstrap treating algorithm family as the resampling unit gives [17.4%, 81.0%]. With eleven families this is uninformative, and we accordingly **do not advance a suite-level rate** (§5.1). The distribution over eligible circuits has **median 0**, $p_{75} = 6.9\%$, $p_{90} = 14.2\%$, $\max = 17.3\%$.

The highest-risk eligible circuits:

circuit	θ	95% CI	risk	95% CI
<code>bv_n280</code>	+4.89%	[+3.67, +6.11]	17.31%	[11.60, 23.69]
<code>knn_n67</code>	+6.67%	[+5.83, +7.51]	16.70%	[10.83, 23.81]
<code>swap_test_n83</code>	+6.76%	[+6.02, +7.50]	15.67%	[10.19, 22.75]
<code>adder_n118</code>	+6.47%	[+5.70, +7.21]	12.67%	[8.16, 19.25]
<code>knn_341</code>	+6.20%	[+5.69, +6.71]	4.68%	[2.70, 7.68]

4.3 The ambiguity band — free of θ , not free of the version pair

The risk in §4.2 depends on where θ happens to sit. The **ambiguity band** does not: it is the width of the interval of true change over which $P(\text{call})$ runs from 0.05 to 0.95, and contains no θ and no direction.

△ **It is not, however, version-pair-free, and an earlier draft of this paper wrongly called it that.** The band is computed by sweeping a synthetic change over the *measured per-seed residual* of the 1.4.3 → 2.0.0 pair, so its shape is inherited from that pair even though its location is not. It is also computed under a **multiplicative** residual model. △ v3 correction: earlier versions said an additive

model “fits better on 29 of 39 circuits”. Recomputation gives **23 additive, 3 multiplicative and 13 ties**. Recomputing the band additively moves the median from **10.87 pp to 10.73 pp**, and only 1 of the 23 resolved stochastic circuits is wider under the additive model. Both figures are computed with the synthetic candidate **truncated to integers**, matching the observable; without truncation they are 10.869 pp and 10.680 pp. The ~11 pp scale survives every one of these constructions, but the conventions are modelling choices and are stated here rather than buried.

△ A second v3 correction, to the horizontal axis. The sweep parameter is *not* exactly the ratio-of-means change θ . Normalising the candidate arm by $m = \text{mean}(B/A)$ makes the realised ratio-of-means multiplier $C \cdot r$ with $C = (\text{mean } B / \text{mean } A) / m$, not r . For `cc_n64`, $C = 0.99104$ and its 25.134 pp parameter width is 24.909 pp in mean-change units; the median converted to mean-change units is 10.852 pp. The difference is small at the headline, but the parameter is not identically θ and earlier versions implied it was.

Among the 23 stochastic circuits, the median ambiguity band is 10.9 percentage points ($p75 = 14.8$, $\text{max} = 25.2$).

Under the stated empirical residual model, this version pair, $k = 3$ and the +10% threshold, the median 5%-to-95% transition width across the 23 resolved stochastic circuits is approximately **10.9 percentage points**. △ v4 correction: earlier versions called such a window “unresolvable”. It is not an impossibility — it is the width over which this detector’s call probability runs from 5% to 95%, so verdicts inside it are substantially decided by the draw rather than by the change. The endpoints need not be symmetric about the threshold. We claim this for the circuits, topology, SDK and version pair measured here; extending it to other version pairs requires measuring their residuals.

This also disposes of the objection that the effect is mere threshold proximity. `cc_n32` has $\theta = -8.28\%$, **18.3 percentage points below** the +10% threshold, and is nonetheless called a regression with probability **0.374089%** — computed exactly as $239,417,152,405 / 64,000,000,000,000$ over the full draw space, not estimated from a sample. Its ambiguity band is 24.5 points. A single-run comparison of the same circuit can return +49.5%.

△ v3 correction, two defects in one sentence. Earlier versions said this risk “rests on 6 of 1728 sampled triple-pairs”. **No such sample exists**: 1728 is the triple count and $0.00374089 \times 1728 = 6.464$, which a checker rounded to 6. The effective-count framing is withdrawn. Earlier versions also attached the interval $[-9.13, -6.05]$ to θ ; that interval targets $\text{mean}(B/A - 1)$, a different estimand from the ratio of arm means reported here. The bootstrap interval for the declared estimand is **$[-9.79\%, -6.77\%]$** . Neither correction changes the fact that this circuit’s empirical risk is non-zero.

4.4 Issue #14402

Two of the three circuits named in #14402 fall in the blind pre-registered sample, chosen by the cost rule: `bv_n280` (risk 17.31%) and `knn_341` (risk 4.68%). The third, `bv_n140`, was measured separately at 400 seeds across 21 processes: $\theta = +5.374\%$ [$+4.272, +6.495$] on heavy-hex, risk **24.4%** [19.5, 31.1]. **It is not part of the pre-registered endpoint.**

We reproduce the issue’s reported figures where seed spread is small, and fail to where it is large — the pattern our own account predicts.

△ v3 correction: earlier versions added that this “is not a Benchpress version difference, which would not be selective”. **That inference is invalid and is withdrawn.** A version change can alter one circuit and leave others untouched, so selectivity excludes nothing. No controlled comparison of the historical and pinned Benchpress revisions was run, and none of the issue’s revision, circuit hashes,

configuration or aggregation rule is available to us. Attributing the mismatch would require that experiment; we do not attribute it.

All three “ours” values below are `linear` topology, 12 seeds per arm, Qiskit 1.4.3 → 2.0.0, optimization level 2, ratio of arm means — not the heavy-hex primary study:

circuit	issue	ours	difference	seed spread
bv_n280	+44.245%	+44.533%	+0.29 pp	0.6%
knn_341	+44.060%	+43.426%	−0.63 pp	1.0%
bv_n140	+46.142%	+35.207%	−10.94 pp	44.4%

△ v3 correction: earlier versions quoted the issue as reporting +44.0%, **+41.0%** and +46.1%, giving differences of +0.5, **+2.4** and −10.9 pp. The issue’s actual figures are those above; the `knn_341` value was misquoted and its difference has the wrong sign. The corrected differences are +0.29, −0.63 and −10.94 pp.

On `linear`, `bv_n140`’s reference change is +31.0% [+28.4, +33.8] at 200 seeds per arm. A single three-run comparison of that circuit — three runs per version, the aggregation the issue reports, simulated **at our configuration** rather than under its own unavailable revision and aggregation rule — has exact support running from **−17.68% to +109.17%**, with a 95% range of [+9.28%, +57.31%]. The issue’s +46.14% sits at the **87.8th** percentile of that distribution, and the same range extends below the +10% threshold. △ v3 correction: earlier versions reported −10.5% to +100.0%, which were extrema of a finite *sample*, not the support bounds. Sampling with replacement permits repeating the minimum or maximum draw three times, so the true support is wider.

4.5 Sensitivity

Threshold. Some risk remains at every cut tested, which is the claim this section supports. △ v3 correction: the percentages previously tabulated here were **not** the pre-registered endpoint. `followup.py` counts circuits whose Monte-Carlo risk estimate exceeds zero, not those whose bootstrap lower bound exceeds zero, and it uses a different θ -bootstrap stream and budget. At the 20% cut its 300,000-pair Monte Carlo reports 18 of 39 and misses six circuits with genuinely positive risk; the exact point-positive count is **24 of 39**, and a 400-resample exact-inner interval analysis gives **22 of 39**. The weak conclusion survives — risk does not vanish at any tested threshold — but the figures are a different statistic from the endpoint and the Monte Carlo has a detection floor.

Runs per version. On `bv_n140`/heavy-hex, pooled over 400 seeds (Monte Carlo, $4 \times 50\text{M}$ samples per point; spread across seeds ≤ 0.012 pp):

<i>k</i>	1	3	5	8	10	20
risk	34.58%	24.38%	18.56%	12.94%	10.36%	3.74%

Monte-Carlo standard error ≤ 0.02 pp per entry; the final digit of each is not significant and should not be quoted alone.

Twenty runs per version — 20×2 versions $\times \sim 2$ h per suite run, so on the order of 80 compute hours — still leaves 3.74%. △ v3 correction: earlier versions said 40 hours, which halves the arithmetic; 40 hours would be wall-clock on two machines running concurrently, not the compute total. The ~ 2 h per-run figure is the issue’s, and we could not independently confirm it. By contrast, one `seed_transpiler` argument removes the **seed-attributable** sampling variance at $k = 1$. △ v4 correction (Astra V4-05): earlier versions said it removes the sampling variance, unqualified. The backend is

constructed without a seed on the same path, and we probed that at 24 cases and found no effect on the observable — a limited negative result, not a demonstration that no other source of variance remains.

Aggregation. Under minimum-of-3 rather than mean-of-3, all eight tested circuits still err and every rate roughly doubles (`bv_n280` 17.3% $\rightarrow$ 28.4%). Best-of- k practice does not remove the problem.

Sampling design. A disjoint set of 200 scattered seeds across ten fresh processes reproduces `bv_n140`/heavy-hex at 22.59% [16.37, 30.61] against 26.28% [18.45, 34.20] from the contiguous single-process set; each point falls inside the other’s interval.

5. Threats to validity

5.1 What we withdraw

An earlier draft of this work reported a suite-level rate. **We withdraw it.** With eleven algorithm families and near-perfect within-family homogeneity, the effective sample size is far below 26 and no interval we can compute is informative about a population of circuits. The pre-registered 12/26 stands as a *descriptive count of these 26 circuits*.

We also withdraw a reported correlation of $\rho = +0.876$ between “compilation is stochastic” and risk. Circuits with both arms constant have zero risk by arithmetic, so a correlation computed over a sample containing them measures a definition, and Spearman with 13 ties at zero is the wrong statistic. Only the $\rho = -0.833$ among stochastic circuits is empirical.

5.2 Limitations

1. **One SDK, one version pair, one topology, one machine.** The 200-seed measurements were all produced on a single machine; only the six-circuit determinism control was repeated on a second one, and both ran the same rustworkx build (§3.4). No other SDK was measured. Source inspection covers two of the eight gyms: the Qiskit gym passes no compiler seed, and the BQSKit gym passes `seed=0`. The remaining six use their own compilation interfaces and are **not assessed here** (§2.1).
2. **0 is a plug-in estimate** from 200 seeds, not an external criterion (§3.1) — and those 200 are the smallest of 400 draws, so they do not represent the full seed range (§3.4). Every result here is conditional on the recorded distributions.
3. **The threshold is ours.** Benchpress defines none; #14402 states no formal cut.
4. **Selection favours fast circuits**, which correlates with small, though the sample spans 28–420 qubits. The variance test in §3.4 fails to reject; that is not evidence of independence, and exploratory results preceded the freeze (§3.3).
5. **The boundary exclusion defines a restricted estimand.** Including boundary circuits raises the endpoint to 22/36; we report the pre-registered figure.
6. **cc_n32’s risk is small and exactly computed**, 0.374089% over the full draw space. The “6 of 1728 triples” framing in v1–v2 is withdrawn: no such sample exists (§4.4).
7. **Pairing is not a universal remedy, and a fixed seed is not a guarantee.** Passing `seed_transpiler` was worse on three of four circuits exhibiting false negatives, and paired mean-of-3 risk remains 17.28% on `bv_n280` and 17.52% on `knn_n67`. A fixed seed makes a verdict repeatable, not correct (§6).
8. **No causal attribution.** The 1.4.3 $\rightarrow$ 2.0.0 comparison spans two major releases and we isolate no mechanism within the compiler.

5.3 Adversarial review

The result was attacked in three rounds by two independent language models acting as hostile reviewers, and by a first-principles audit. That audit returned **zero invalidating findings, three rewordings — two of which are §5.1 — and two limitations**. A later independent audit of the *code* rather than the claims found six implementation defects, none of which changed a reported number; they are recorded in `SETTLED.json` as S17–S22.

`inventory.py --check` compares all **44** recorded figures against a fresh recomputation: **32 are re-derived from the raw per-seed measurements (RAW-RECOMPUTED)**, and **12 — the sec 43 k-sweep — are re-read from a derived summary table (DERIVED-READ)**, each row labelled with which tier it belongs to. $\triangle$ v4 correction: v1–v3 said 41/29/12, which was stale once the primary endpoint was registered here in v3. Re-deriving the k-sweep from raw means enumerating 12^5 mean tuples per bisection step, roughly half an hour per topology, which is not affordable inside a check that has to run in under a minute. **We therefore do not claim that every reported number is re-derived from raw data**, and the tool says so in its own documentation. The repository records every withdrawn claim, including four from earlier phases of the work.

6. Conclusion

Unseeded stochastic compilation makes regression verdicts probabilistic, and the probability is measurable. On a benchmark used to evaluate a production quantum compiler, a three-run comparison’s call probability runs from 5% to 95% across a window of roughly eleven percentage points of true change for the median stochastic circuit — so within that window the verdict is substantially decided by the draw. $\triangle$ v3 correction: earlier versions said such a change “cannot be resolved”, which states an impossibility the construction does not support; the band is a transition region of a particular detector’s call probability, not a confidence interval or a theorem, and its endpoints need not be symmetric about the threshold. The two circuits from the motivating bug report that we could evaluate blind both carry non-zero risk. Adding runs reduces the risk slowly — on the demonstrated circuit, twenty runs per version still leaves 3.74% — while setting a seed removes this source of sampling variance outright. We report the measured k-sweep rather than fitting a scaling law: a log-log fit to those six points has slope -0.70 , and six points on one circuit do not establish an exponent.

Fixing the seed removes the variation attributable to `seed_transpiler`. $\triangle$ v3 correction: it does **not** guarantee a correct verdict, and earlier versions came close to saying so. A fixed seed makes the answer repeatable, including when that answer disagrees with the long-run mean — on `bv_n280`, the recorded seed 663193 gives $A = 1040$ against $B = 1157$, a +11.25% call against a measured θ of +4.89%. Frozen, reproducible, and the wrong side of the threshold. Pairing seeds across arms does not remove the primary risks either: the paired mean-of-3 risk is 17.28% on `bv_n280` and 17.52% on `knn_n67`. Backend error rates are drawn separately and remain unseeded.

The measurement problem is not exotic — it is the ordinary consequence of treating a stochastic measurement as a scalar. We suggest that benchmark suites for stochastic compilers should either fix seeds **and report that the verdict is conditional on them**, or report the distribution and the decision risk alongside any verdict.

6.1 The apparatus, and what it would take to generalise it

What we built is not specific to Qiskit. Given a benchmark with a scalar observable, a enumerable source of run-to-run variation, and a decision rule, the same pipeline yields a decision-risk estimate and

an ambiguity band. Four components carried the weight and are reusable:

1. **Toolchain pinning that includes the benchmark itself.** Our provenance records named only the compiler until an audit found it; the benchmark supplies the circuits, the backend, the topology and the observable, and is equally part of the measurement.
2. **A pre-registration mechanism with a verifiable commit order.** Committing the circuit list and the analysis code before the primary data exists makes the freeze auditable. Δ v3 correction: it does not convert “we did not cherry-pick” into a fact. It records that the design was fixed before *this* data arrived, not that no earlier exploration informed it — and here it did (§3.3).
3. **A numeric inventory, and a verifier that has been shown to reject us.** The inventory carries **44** recorded values with their numerator, denominator, sampling unit, interval, method, and a function that recomputes each — **32** recomputed from raw data and **12** re-read from derived k-sweep CSVs. Δ v4 correction (Astra V4-05): earlier versions said “every published figure”. They do not cover every figure, every interval, or any prose, and the distinction between a raw recomputation and a re-read of a derived file matters; `python inventory.py --check` prints the 44/32/12 split.

Δ v3 correction, and the most important one in this paper. v1 and v2 claimed that “a single command re-checks the whole paper” and that “a stale or altered source fails the check”. **Both were false, and a hostile audit proved it.** Replacing every candidate gate count in one raw arm file with a constant changes the true endpoint from 12/26 to 11/26 and the $\geq 5\%$ and $\geq 10\%$ counts from 7/26 and 4/26 to 6/26 and 3/26 — and `verify.py` still reported **6/6 PASS**. The cause was structural: the primary numbers were read from a derived summary CSV, and a summary cannot notice that the raw data beneath it changed.

The verification chain now begins at the raw per-seed observations and reconstructs θ , the verdict, boundary status, eligibility, the risk and its interval, the 12/26, 7/26 and 4/26 counts, and finally the literals printed here — reading no summary table, and using a risk estimator written independently of the one under test. Raw evidence is additionally hashed against a committed manifest, and the derived summary is required to be reproducible **from** raw rather than trusted.

The claim we now make is narrower and is itself tested: `mutation_test.py` corrupts data in throwaway snapshots at three different layers and **requires** the relevant stage to fail. A verifier that only ever agrees with its authors is worthless; the evidence that this one can turn red is a test we run, not an assurance we offer.

4. **Adversarial review as a required stage, not an optional one.** Four rounds of hostile review plus two first-principles audits removed five claims from this work and corrected many more, including the verification claim in item 3 — which survived three earlier rounds before an auditor thought to corrupt the data instead of the prose.

The obvious next step is more algorithm families and more SDKs; the sample here is underpowered for population-level statements precisely because the outcome is homogeneous within families. That is an experiment, not an analysis, and we do not pre-empt it.

Data and code availability

All raw per-seed measurements, analysis code, the pre-registration, the defect record and the complete research log are in the `qvalidation` repository. `verify.py` re-runs the toolchain pin check, test suite, numeric inventory, replication artifact and a proof of a withdrawn analytical claim. `inventory.py --`

check compares all 44 recorded numbers against a fresh recomputation, 32 of them re-derived from the raw per-seed files and 12 re-read from a summary table, each row labelled with which.

References

1. Nation, P. D. et al. Benchmarking the performance of quantum computing software. *Nature Computational Science* (2025). doi:10.1038/s43588-025-00792-y
2. Qiskit issue #14402, *Significant increase in 2Q gate counts for many Benchpress tests when going from Qiskit 1.4 to 2.0*. <https://github.com/Qiskit/qiskit/issues/14402>
3. Li, G., Ding, Y. & Xie, Y. Tackling the qubit mapping problem for NISQ-era quantum devices. *ASPLOS* (2019). — SABRE
4. Pati, A. & Simmhan, Y. On the reproducibility of quantum circuit transpilation. arXiv:2605.07876